POO EN PYTHON II

4. EXCEPCIONES.

Aunque una declaración o expresión sea sintácticamente correcta, puede generar un error cuando se intenta ejecutar. Los errores detectados durante la ejecución se llaman **excepciones** y se pueden gestionar.

Python trata todos los errores de la misma forma, como excepciones, pero utiliza dos categorías para indicar la severidad del error: Excepciones y Warnings. Las primeras deben ser capturadas y tratadas y para las segundas se puede configurar el sistema para ignorarlas. El mecanismo usado para controlar la gestión de los warnings es el filtro de advertencias, que controla si las advertencias se ignoran, se muestran o se convierten en errores (planteando una excepción). Se definen los siguientes warnings.

Clase	Descripción
Warning	Esta es la clase principal para todas las clases que pertenecen a la categoría de advertencia. Es una subclase de Exception.
UserWarning	La categoría por defecto para warn().
DeprecationWarning	Categoría principal para advertencias sobre características obsoletas cuando esas advertencias están destinadas a otros desarrolladores de Python (ignoradas por defecto, a menos que sean activadas por el código en __main__).
SyntaxWarning	Categoría principal para las advertencias sobre características sintácticas dudosas.
RuntimeWarning	Categoría principal para las advertencias sobre características dudosas de tiempo de ejecución.
FutureWarning	Categoría principal para las advertencias sobre características obsoletas cuando esas advertencias están destinadas a los usuarios finales de aplicaciones escritas en Python.

POO EN PYTHON II

4. EXCEPCIONES.

Clase	Descripción
PendingDeprecationWarning	Categoría principal para las advertencias sobre las características que serán desaprobadas en el futuro (ignoradas por defecto).
ImportWarning	Categoría principal para las advertencias que se activan durante el proceso de importación de un módulo (se ignoran por defecto).
UnicodeWarning	Categoría principal para las advertencias relacionadas con Unicode.
BytesWarning	Categoría principal para las advertencias relacionadas con bytes y bytearray.
ResourceWarning	Categoría base para las advertencias relacionadas con el uso de los recursos.

POO EN PYTHON II

4. EXCEPCIONES.

Jerarquías

En Python, todas las excepciones deben ser instancias de una clase que se derive de **BaseException**. En una instrucción **try** con una cláusula **except** que menciona una clase determinada, esa cláusula también controla las clases de excepción derivadas(hijas), es decir, si capturamos una excepción también se capturarán todas sus hijas si no lo hacemos explícitamente antes. Dos clases de excepción que no están relacionadas mediante subclases (hermanas o herederas de hermanas) nunca son equivalentes, incluso si tienen el mismo nombre.

```
BaseException
 +-- SystemExit
 +-- KeyboardInterrupt
 +-- GeneratorExit
 +-- Exception
      +-- StopIteration
      +-- StopAsyncIteration
      +-- ArithmeticError
      |    +-- FloatingPointError
      |    +-- OverflowError
      |    +-- ZeroDivisionError
      +-- AssertionError
      +-- AttributeError
      +-- BufferError
      +-- EOFError
      +-- ImportError
      |    +-- ModuleNotFoundError
      +-- LookupError
      |    +-- IndexError
      |    +-- KeyError
      +-- MemoryError
      +-- NameError
      |    +-- UnboundLocalError
      +-- OSError
      |    +-- BlockingIOError
      |    +-- ChildProcessError
      |    +-- ConnectionError
      |    |    +-- BrokenPipeError
      |    |    +-- ConnectionAbortedError
      |    |    +-- ConnectionRefusedError
      |    |    +-- ConnectionResetError
      |    +-- FileExistsError
      |    +-- FileNotFoundError
      |    +-- InterruptedError
      |    +-- IsADirectoryError
      |    +-- NotADirectoryError
      |    +-- PermissionError
      |    +-- ProcessLookupError
      |    +-- TimeoutError
      +-- ReferenceError
      +-- RuntimeError
      |    +-- NotImplementedError
      |    +-- RecursionError
      +-- SyntaxError
      |    +-- IndentationError
      |         +-- TabError
      +-- SystemError
      +-- TypeError
      +-- ValueError
      |    +-- UnicodeError
      |         +-- UnicodeDecodeError
      |         +-- UnicodeEncodeError
      |         +-- UnicodeTranslateError
      +-- Warning
           +-- DeprecationWarning
           +-- PendingDeprecationWarning
           +-- RuntimeWarning
           +-- SyntaxWarning
           +-- UserWarning
           +-- FutureWarning
           +-- ImportWarning
           +-- UnicodeWarning
           +-- BytesWarning
           +-- ResourceWarning
```

POO EN PYTHON II

4. EXCEPCIONES.

Capturas

Es posible escribir programas que gestionen determinadas excepciones. Véase el siguiente ejemplo, que le pide al usuario una entrada hasta que ingrese un entero válido, pero permite al usuario interrumpir el programa (usando Control-C o lo que soporte el sistema operativo); nótese que una interrupción generada por el usuario es señalizada generando la excepción **KeyboardInterrupt**.

```
def solicitar_entero():
    while True:
        try:
            entrada = input("Ingrese un número entero: ")
            numero = int(entrada)  # Intenta convertir la entrada en entero
            print(f"Número válido ingresado: {numero}")
            break  # Salimos del bucle si la conversión es exitosa
        except ValueError:
            print("Error: No es un número entero válido. Intente de nuevo.")
        except KeyboardInterrupt:
            print("\nInterrupción detectada. Saliendo del programa...")
            break  # Permite salir del bucle de forma controlada

# Llamamos a la función
solicitar_entero()
```

POO EN PYTHON II

4. EXCEPCIONES.

Capturas

La declaración try funciona de la siguiente manera:

- Primero, se ejecuta la cláusula try (la(s) linea(s) entre las palabras reservadas **try** y la **except**).
- Si no ocurre ninguna excepción, la cláusula **except** se omite y la ejecución de la cláusula try finaliza.
- Si ocurre una excepción durante la ejecución de la cláusula try el resto de la cláusula se omite. Entonces, si el tipo de excepción coincide con la excepción indicada después de la **except**, la cláusula **except** se ejecuta, y la ejecución continua después de la try.
- Si ocurre una excepción que no coincide con la indicada en la cláusula **except** se pasa a los **try** más externos; si no se encuentra un gestor, se genera una unhandled exception (excepción no gestionada) y la ejecución se interrumpe con un mensaje.

Una declaración **try** puede tener más de un **except**, para especificar gestores para distintas excepciones, Una clase en una cláusula **except** es compatible con una excepción si es de la misma clase o de una clase derivada de la misma por lo que el orden de las sentencias **except** es muy importante.

POO EN PYTHON II

4. EXCEPCIONES.

Capturas

Puede existir un último **except** omitiendo el nombre de la excepción capturada y servir como comodín para capturar cualquiera no indicada, así como un bloque **else** opcional, el cual, cuando está presente, debe seguir a los **except** y se ejecutará si el código dentro de try no genera errores.

```
for arg in sys.argv[1:]:
    try:
        f = open(arg, 'r')
    except OSError:
        print('cannot open', arg)
    else:
        print(arg, 'has', len(f.readlines()), 'lines')
        f.close()
```

La cláusula **except** puede especificar una variable después del nombre de excepción para recoger los datos de la excepción y tratarla de forma adecuada.

```
try:
    raise Exception('spam', 'eggs')
except Exception as inst:
    print(type(inst)) # the exception instance
    print(inst.args) # arguments stored in .args
    print(inst) # __str__ allows args to be printed directly,
    x, y = inst.args     # unpack args
    print('x =', x)
    print('y =', y)
```

POO EN PYTHON II

4. EXCEPCIONES.

Capturas

Para finalizar se puede incluir un bloque **finally** que se ejecutará siempre en última instancia.

```
try
    …
} finally {
    # bloque que siempre se ejecutará el último
    # independientemente de si hay o no excepción
    # después del bloque else si se tiene que ejecutar
}
```

Los gestores de excepciones no se encargan solamente de las excepciones que ocurren en el bloque try, también gestionan las excepciones que ocurren dentro de las funciones que se llaman (inclusive indirectamente) dentro del bloque try.